From Encoder–Decoder to ChatGPT: A History of Large Language Models

Contents

1. Overview 2
2. Prerequisites 3
3. Sequence-to-sequence problems 3
4. Stage 1: the encoder–decoder (2014) 4
5. Stage 2: attention (2014–15) 5
6. Stage 3: the transformer (2017) 7
7. Stage 4: transfer learning for language (2018) 8
8. Stage 5: large language models (2018 onward) 9
9. From GPT-3 to ChatGPT 12
10. Summary 15
11. Sources 15
12. Key terms 17

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Used here, taught in full later:** Transformer; Decoder-only GPT.

1. Overview

Key point: Large language models (G-1046) grew out of five stages, each fixing the main problem of the one before; training GPT for dialogue with human feedback then gave **ChatGPT**.

The five stages:

1. The **encoder–decoder** (2014) turned one sequence into another but forgot long sentences.
2. **Attention** (G-226) (2014–15) let it look back at every input word, but it was slow because it still read one word at a time.
3. The **transformer** (G-2007) (2017) dropped the RNN and read all words in parallel, but it needed huge data.
4. **Transfer learning** (G-2005) (2018) removed that need: pre-train once on unlabelled text, then fine-tune.
5. Combining transformers with transfer learning at a vast scale gave **LLMs** such as GPT-3.

The last block of the deep learning Notes works with **sequence-to-sequence** models, the family behind **machine translation** (G-1141), chatbots and ChatGPT. This Note is the map for that block. It tells the story stage by stage: what each stage invented, who invented it, and what problem it left for the next stage. In Figure 1, watch the stages appear one at a time: each red line is the problem a stage left, and the next box is the idea that fixed it. The architectures themselves each get their own Notes later:

- the encoder–decoder;
- **attention** (G-226);
- self-attention;
- the transformer.

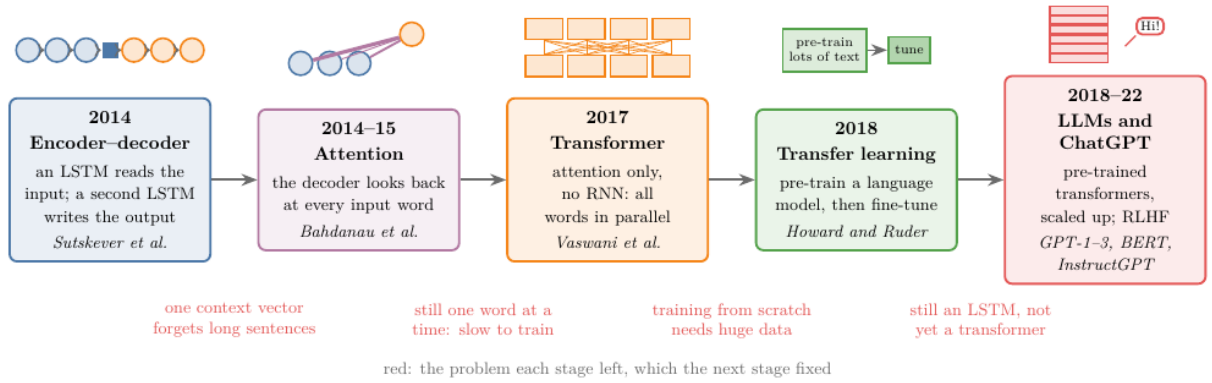


Figure 1: Five stages from the encoder–decoder to ChatGPT, each with a sketch of its architecture above it. Red: the problem each stage left, which the next stage fixed

2. Prerequisites

- Why RNNs are needed: sequential data, and why a plain ANN struggles with it.
- Architectures and transfer learning: **transfer learning** (G-2005) in computer vision.
- The RNN Notes after "Why RNNs are needed", especially the LSTM: a network that reads a sequence step by step and keeps a memory.

3. Sequence-to-sequence problems

> **Key point:** A sequence-to-sequence task takes a sequence in and gives a sequence out, and the two lengths can differ.

RNN tasks come in a few shapes:

- **Sentiment analysis** (G-1769) reads a sequence and outputs one label (many inputs, one output).
- **Image captioning** (G-918) takes one image and outputs a sentence (one input, many outputs).
- **Tagging** each word of a sentence with its part of speech gives one output per input word, so the two sequences have the same length.

The hardest shape has a sequence on both sides with lengths that do not match. Translating "I love India" into Hindi gives "main Bharat se pyaar karta hoon": 3 words in, 6 words out. Tasks of this shape are called **sequence-to-sequence** (seq2seq; G-1772) tasks.

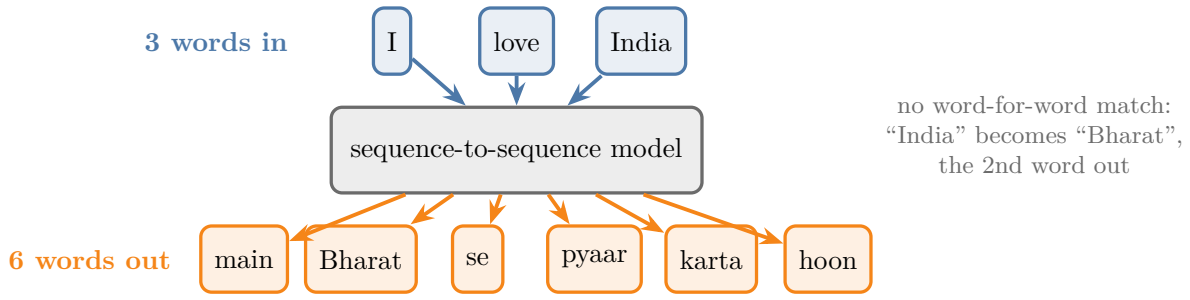


Figure 2: Translation as a sequence-to-sequence task: 3 words in, 6 words out. The output words do not line up one-to-one with the input words

In Figure 2, follow “India”: it becomes “Bharat”, the second output word, so the model cannot simply emit one output word per input word. Examples of such tasks:

- **Machine translation** (G-1141): a sentence in one language in, the same sentence in another language out.
- **Text summarization:** a long text in, a short summary out.
- **Question answering:** a question in, an answer out.
- **Chatbots:** a message in, a reply out.
- **Speech to text:** a sequence of sound values in, a sequence of words out, as in automatic subtitles.

4. Stage 1: the encoder–decoder (2014)

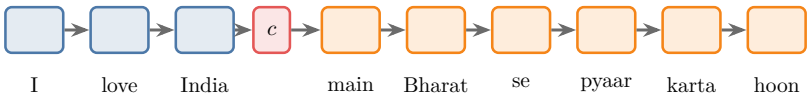
> **Key point:** One LSTM, the encoder, reads the input word by word and squeezes it into a single **context vector** (G-461; a list of numbers); a second LSTM, the decoder, writes the output word by word from that vector. It works on short sentences and fails on long ones.

In 2014 Ilya Sutskever, Oriol Vinyals and Quoc Le at Google published “Sequence to Sequence Learning with Neural Networks” (Sutskever et al. 2014). Sutskever later co-founded OpenAI (OpenAI 2015). Their model has two parts (Figure 3, top):

1. **Encoder** (G-682): an LSTM reads the input sentence one word per step. Its internal state is updated at every step, so after the last word the state is a summary of the whole sentence. This summary is the **context vector**.
2. **Decoder** (G-564): a second LSTM starts from the context vector and produces the output sentence one word per step.

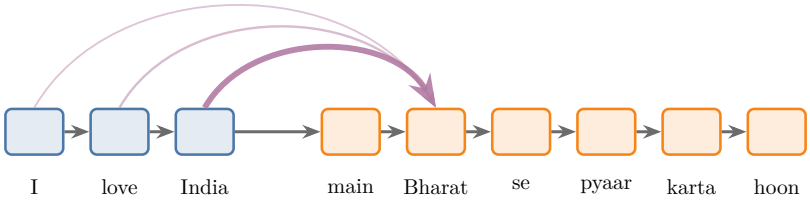
A simple RNN or GRU cell would also work inside each part; the paper used the **LSTM** (G-1123).

(a) Encoder–decoder: one context vector



encoder (blue) squeezes the whole sentence into c; the decoder (orange) sees only c

(b) With attention: every input word stays visible



to write "Bharat" the decoder weighs all encoder states; the thickest line is to "India"

Figure 3: Top: the encoder (blue) squeezes the sentence into one context vector c, and the decoder (orange) sees only c. Bottom: with attention, the decoder can look back at every encoder state; the line widths show how much each input word counts for the current output word

The weak point is the context vector. However long the input, everything must pass through one fixed-size vector. An analogy: reading a whole paragraph once and then translating it from memory. A short sentence fits in memory; a long paragraph does not, and the start is the first part to fade.

The measurements agree. Bahdanau, Cho and Bengio (2015, Figure 2) plotted translation quality, measured by the **BLEU score** (G-315) (how many word sequences of a translation match a human reference translation), against sentence length. For the plain encoder–decoder, quality "dramatically drops as the length of the sentences increases". Our own data shows the same drop for the plain encoder–decoder: its BLEU falls from 14.3 on 1–4-word sentences to 5.9 on 14–16-word sentences (attention against no attention by sentence length). The **attention** (G-226) model of the next section loses less on the same sentences.

5. Stage 2: attention (2014–15)

> **Key point:** Instead of one context vector, the **decoder** (G-564) gets a fresh context vector for every output word: a weighted mix of all the encoder's states, with the weights learned. The decoder can therefore always look at the right input words. The price is speed.

Dzmitry Bahdanau, Kyunghyun Cho and Yoshua Bengio proposed the fix in "Neural **Machine Translation** (G-1141) by Jointly Learning to Align and Translate" (on arXiv in 2014, published at ICLR 2015). Their abstract names the problem directly: the encoder–decoder

"needs to be able to compress all the necessary information of a source sentence into a fixed-length vector", which makes long sentences hard.

Attention keeps every state the **encoder** (G-682) produced, one per input word. When the decoder is about to write an output word, a small neural network, trained together with the rest, scores how useful each encoder state is for that word. The scores become weights, and the weighted mix of encoder states becomes the context vector for that one step (Figure 3, bottom). When the decoder writes "Bharat", most of the weight goes to the state for "India".

Because no word has to survive a long trip through one vector, the beginning of a long sentence is no longer lost. In the same Figure 2 of Bahdanau et al., the **attention** (G-226) model trained on sentences of up to 50 words shows "no performance deterioration" as sentences get longer.

Attention has two costs:

1. **More computation.** With n input words and m output words, the model scores every input word for every output word: $n \times m$ scores per sentence pair. Doubling both lengths quadruples the work (Figure 4).
2. **Still sequential.** The encoder and decoder are still **LSTMs** (G-1123), which must process word 1 before word 2 before word 3. Training cannot spread one sentence across many processors at once.

The second cost turned out to be the real limit.

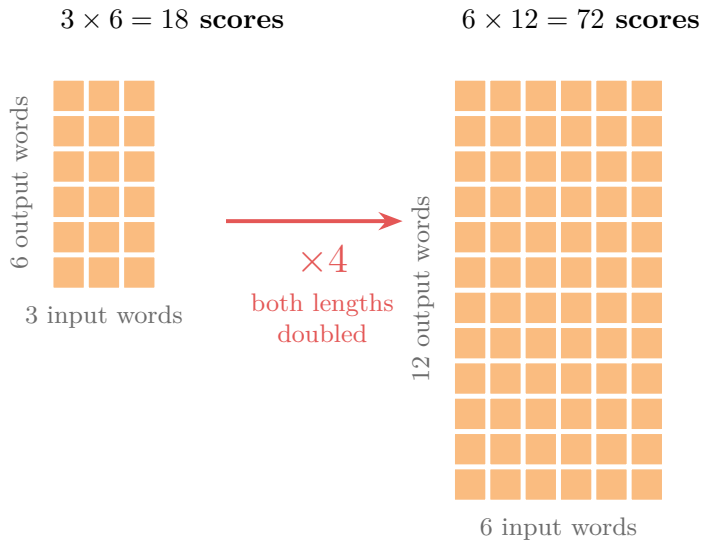


Figure 4: One attention score for every pair of an input word and an output word. A sentence pair of 3 and 6 words needs 18 scores; doubling both lengths gives 6 and 12 words and 72 scores

6. Stage 3: the transformer (2017)

> **Key point:** The **transformer** (G-2007) removes the RNN completely and uses attention alone. Every word of the input is processed at the same time, so training runs in parallel and is far faster.

In 2017 a team at Google Brain and Google Research published "**Attention** (G-226) Is All You Need" (Vaswani et al. 2017). The paper points to the bottleneck: an RNN's "inherently sequential nature precludes parallelization within training examples". Their **transformer** keeps the encoder–decoder shape but contains no **LSTM** (G-1123). It is built from parts the earlier Notes already know or will introduce soon: attention (in a new form called **self-attention** (G-1763), where the words of one sentence attend to each other), plus three parts:

- **dense layers** (every node connected to every input, as in the MNIST network);
- **normalization layers** (they rescale each word's numbers to a standard range; see normalisation in deep learning);
- **embeddings** (each word turned into a list of numbers; see turning words into numbers).

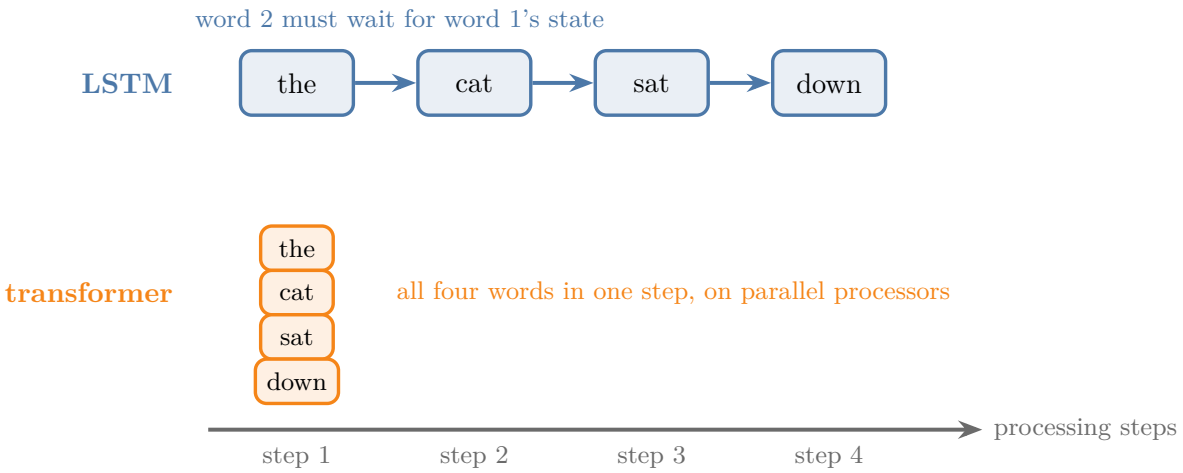


Figure 5: Top: an LSTM processes one word per step, because each word needs the state left by the word before. Bottom: a transformer processes all the words of the sentence in the same step

Figure 5 shows the difference in processing steps: four steps for four words in the LSTM, one step in the transformer.

Because the transformer sees all words of a sentence at once, the work for one sentence can be split across many processors. The paper reports better translation scores than earlier models "at a fraction of the training cost" (Vaswani et al. 2017, Table 2).

One problem remained. Trained from scratch, a transformer needs a great deal of labelled data, computing power and time. A small company with a few thousand labelled reviews

could not train one well.

7. Stage 4: transfer learning for language (2018)

> **Key point:** Pre-train a model once on a huge amount of unlabelled text by teaching it to predict the next word, then fine-tune it on a small labelled dataset for the task at hand. With 100 labelled examples, this matched training from scratch on 100 times more data.

Transfer learning reuses knowledge learned on one task for a related task: someone who can ride a bicycle learns to ride a motorbike faster. In computer vision it was already standard (see architectures and transfer learning). It works in two steps:

1. **Pre-training:** train a model on a huge general dataset, such as the millions of images of ImageNet, so that it learns general features such as edges and shapes.
2. **Fine-tuning:** keep the early layers, replace the last ones, and train on our own small dataset, for example 100 photos of cats and dogs.

For text, **transfer learning** (G-2005) had not taken hold. One reason was the choice of pre-training task: translation was tried, but it needs pairs of sentences in two languages, which are scarce.

In January 2018 Jeremy Howard and Sebastian Ruder published ULMFiT, "Universal Language Model Fine-tuning for Text Classification" (Howard and Ruder 2018). Their pre-training task was **language modelling** (G-1042): predicting the next word of a text, as in "the capital of India is ____". Language modelling suits pre-training for two reasons:

- **It teaches a lot.** To predict the next word well, a model must learn grammar, meaning and some facts about the world. In "The hotel was very clean, but the service was ____", only a negative word fits after"but".
- **It needs no labels.** Every piece of text is already its own training data: the next word is the answer. Any amount of unlabelled text can be used. Learning from targets made from the data itself, with no human labels, is called **self-supervised learning** (G-2279): "the natural sequence of words is its own supervision" (Jurafsky and Martin, §7.7). Papers of 2018, GPT among them, still called it *unsupervised* pre-training.

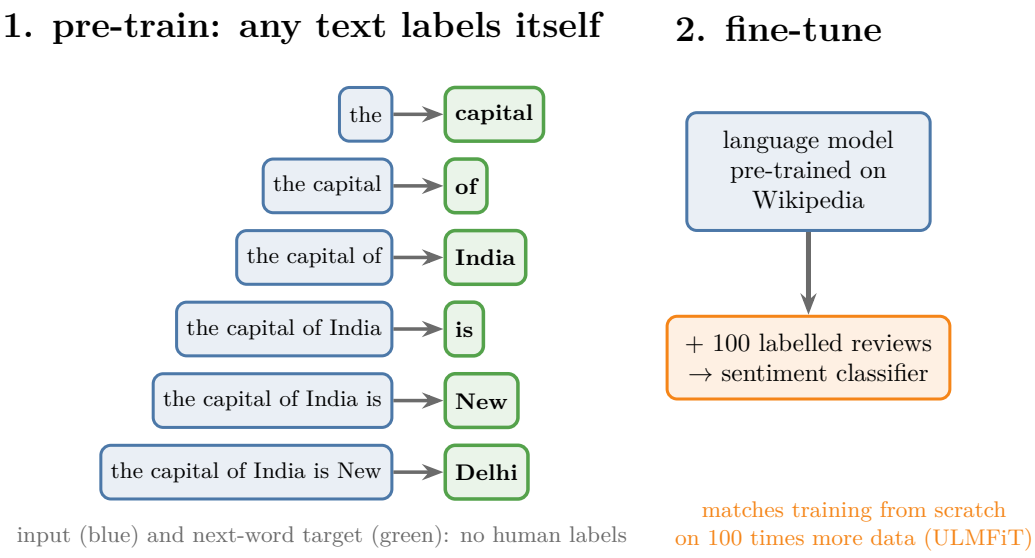


Figure 6: Left: one sentence gives six training examples for language modelling; each prefix is an input and the next word is its target, so no human labelling is needed. Right: the pre-trained model is then fine-tuned on a small labelled dataset

In Figure 6, every green target comes from the sentence itself: that is why any amount of unlabelled text can be used for pre-training.

ULMFiT pre-trained an LSTM-based language model on Wikipedia articles, then fine-tuned it on classification datasets such as IMDB reviews. The paper's headline result: "with only 100 labeled examples, it matches the performance of training from scratch on 100× more data" (Howard and Ruder 2018, abstract).

ULMFiT still used an **LSTM** (G-1123), not a transformer. Joining the two ideas was the next step.

8. Stage 5: large language models (2018 onward)

> **Key point: Transformers** (G-2007) pre-trained as language models on enormous text collections became general-purpose models that can be fine-tuned for almost any language task. Scaled up to billions of parameters (GPT-2, GPT-3), they became **large language models** (G-1046) (LLMs).

The first two pre-trained transformer language models appeared in 2018. With 117 million (GPT) and 340 million (BERT-Large) parameters they were not yet "large"; the scaling came next.

Model	From	Released	Architecture	Pre-training task
GPT	OpenAI	June 2018	decoder only	predict the next word
BERT	Google	October 2018	encoder only	predict hidden (masked) words from both sides

GPT (Radford et al. 2018) was a language model in the ULMFiT sense, but built on a transformer. BERT (Devlin et al. 2019) used a different pre-training task, a **masked language model** (G-1170): some words of the input are hidden, and the model predicts them using the words on both sides. Both could be fine-tuned with little data for **sentiment analysis** (G-1769), question answering, named entity recognition (finding names of people, places and organisations in text) and more, and both set new records. The two keep different halves of the transformer:

- **decoder only** (GPT): reads the text left to right and writes the next word; see from the transformer decoder to GPT;
- **encoder only** (BERT): reads the whole sentence at once, both sides of every word, and writes no new text.

OpenAI kept scaling GPT up. Each new version had many more **parameters** (G-1450) (weights) than the last (Figure 7): 117 million in GPT-1, 1.5 billion in GPT-2 (Radford et al. 2019) and 175 billion in GPT-3 (Brown et al. 2020). At this size people started to say **large** language model.

Figure 7 draws the counts on a **log scale** (an axis where each labelled step up is ten times the one below: 100M, 1B, 10B, 100B, 1T). On an ordinary axis GPT-1's bar would be less than a thousandth of GPT-3's and would not show; the log scale gives each model a visible bar. BERT-Large (orange) is not a GPT model and is shown for comparison.

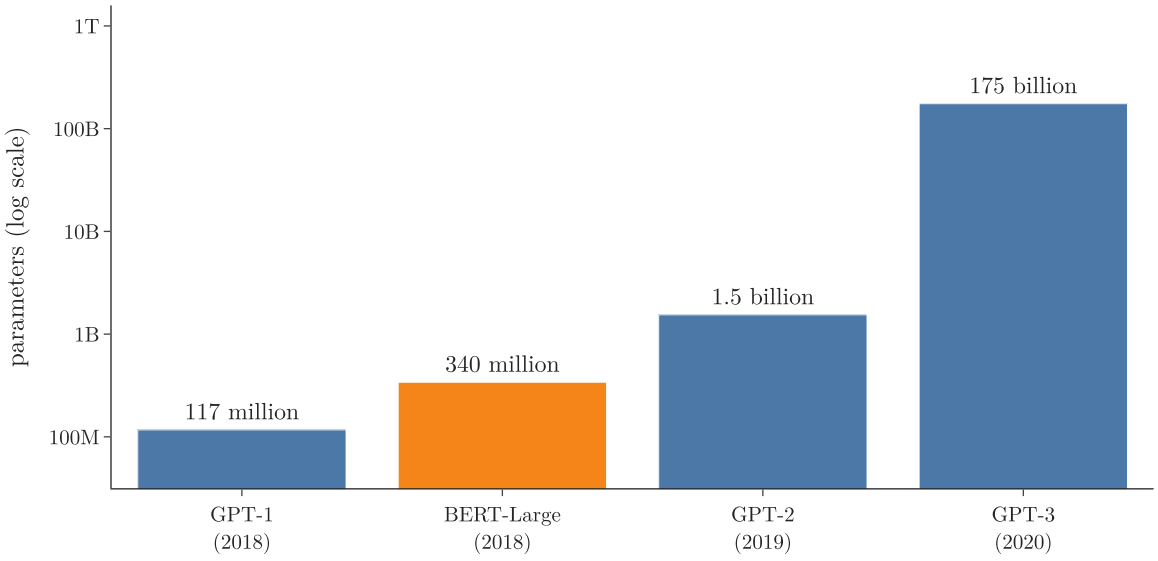


Figure 7: Parameter counts from the original papers, on a log scale: each step up is ten times more. GPT-3 has about 1,500 times the parameters of GPT-1

What makes a language model "large" shows up in five places, here with GPT-3's numbers:

1. **Data.** GPT-3 was trained on 300 billion tokens (words and word pieces). Its largest source, a crawl of the web, was 45 terabytes (45,000 gigabytes) of compressed text before filtering, and 570 gigabytes survived the quality filters (Brown et al. 2020, section 2.2). The share kept:

$$\frac{570}{45{,}000} = 0.013$$

   That is about 1.3 percent. Other sources included books, Wikipedia and web pages linked from Reddit.

2. **Hardware.** GPT-3 was trained on 10,000 NVIDIA V100 GPUs (Patterson et al. 2021, Table 4), "part of a high-bandwidth cluster provided by Microsoft" (Brown et al. 2020).

3. **Time.** Even on 10,000 GPUs, training took about 15 days (14.8 days; Patterson et al. 2021, Table 4). Counted in operations, GPT-3's training took about 3.14×10^{23} floating-point operations (additions and multiplications of decimal numbers; Brown et al. 2020, Table D.1). A computer doing a billion (10^9) operations every second would need:

$$\frac{3.14 \times 10^{23}}{10^9} = 3.14 \times 10^{14} \text{ seconds}$$
$$60 \times 60 \times 24 = 86{,}400 \text{ s per day}$$
$$86{,}400 \times 365 = 3.15 \times 10^7 \text{ s per year}$$

$$\frac{3.14 \times 10^{14}}{3.15 \times 10^{7}} \approx 10^{7} \text{ years}$$

That is about **10 million years** (Notebook). The 10,000 GPUs did it in two weeks only because each one did about 2.5 trillion operations per second on average, all at the same time. One GPU's share, step by step:

$$14.8 \times 86{,}400 = 1.28 \times 10^{6} \text{ s}$$
$$10{,}000 \times 1.28 \times 10^{6} = 1.28 \times 10^{10}$$
$$\frac{3.14 \times 10^{23}}{1.28 \times 10^{10}} \approx 2.5 \times 10^{12}$$

The first line is the training time in seconds (86,400 seconds per day), the second is GPU-seconds, and the third is operations per GPU per second.

(Sanderson 2024, "Large Language Models explained briefly", uses the same yardstick for the largest models).

4. **Cost.** 10,000 GPUs for 14.8 days is a number of GPU-hours:

$$10{,}000 \times 14.8 \times 24 \approx 3.6 \text{ million}$$

That is before counting the people, the buildings and the failed attempts. Only large companies, governments and large research institutes can pay for that.

5. **Energy.** Training GPT-3 used an estimated 1,287 megawatt-hours of electricity and emitted about 552 tonnes of CO_2 (Patterson et al. 2021). An average American home uses about 10.8 megawatt-hours a year (EIA 2023). Dividing gives how many such homes a year of electricity would supply:

$$\frac{1{,}287}{10.8} \approx 119$$

So training GPT-3 used the yearly electricity of about 119 homes.

9. From GPT-3 to ChatGPT

> **Key point:** GPT is the model; ChatGPT is an application built on it. ChatGPT came from fine-tuning a GPT model on dialogue, then improving it with **reinforcement learning from human feedback** (RLHF).

GPT and ChatGPT are often confused. GPT is a model; ChatGPT is a chat application built on a GPT model and released by OpenAI on 30 November 2022 (OpenAI 2022). An analogy: a laptop brand and the processor inside it. We call the laptop by its brand, not by its processor, and the same processor can power other brands' laptops. In the same way, other companies build their own products on GPT models through OpenAI's paid interface.

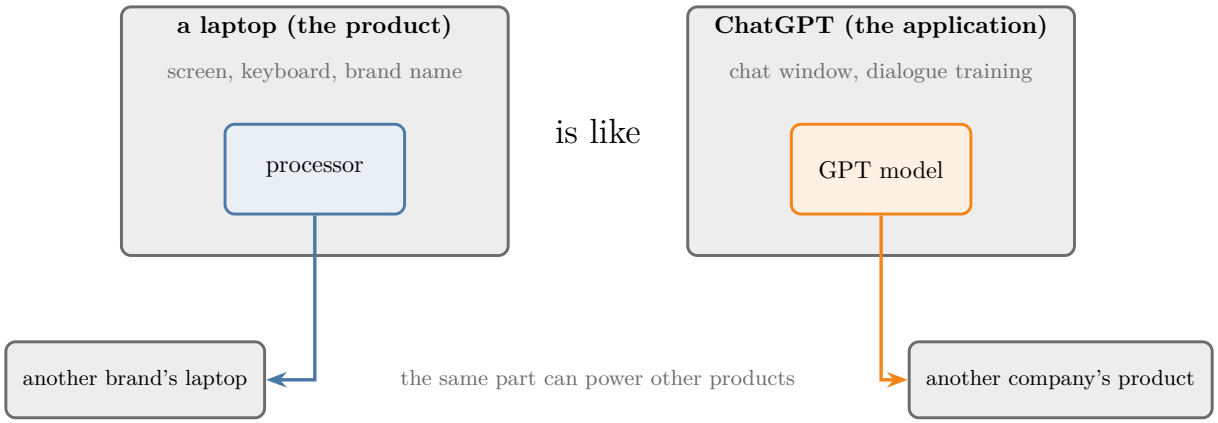


Figure 8: The analogy as a picture. Left: a laptop is a product built around a processor. Right: ChatGPT is an application built around a GPT model. The same processor, or the same model, can also power someone else's product

In Figure 8, the inner box is the part that does the computing and the outer box is what the user sees and names.

A chatbot is a next-word predictor in a script. A language model only continues text. A chat application turns it into an assistant in three steps (Sanderson 2024, "**Large Language Models** (G-1046) explained briefly" and "Transformers, the tech behind LLMs"):

1. **The script.** Write a text around the user's message: a short setup ("What follows is a conversation between a user and a helpful AI assistant"), then the user's words after "User:", then "Assistant:".
2. **The loop.** The model predicts the next word, the word is appended, and the loop repeats.
3. **The reply.** Whatever the model writes after "Assistant:" is shown as the reply.

The format alone is not enough. The Notebook tests it:

- **Experiment:** give exactly this script, with the question "What is the capital of France?", to GPT-2 small, a pre-trained model with no further training, and let it choose the most likely word 30 times.
- **Result:** it writes "Assistant: I'm a French citizen." and then goes on to invent the user's next turn itself.

GPT-2 continues the layout of the text, but it is not a helpful assistant. The training steps below are what make the reply helpful.

ChatGPT was trained with the method of InstructGPT (Ouyang et al. 2022; OpenAI 2022), in three steps:

1. **Supervised fine-tuning.** Human trainers wrote example conversations, playing both the user and the assistant. A GPT model was fine-tuned on these examples, so it learned what a good reply looks like.

2. **A reward model.** The model wrote several replies to the same prompt, and humans ranked them from best to worst. A second model learned to predict these rankings, giving a score for any reply.
3. **Reinforcement learning** (learning by trying replies and being rewarded for good ones). The chat model was trained further to produce replies that score highly with the reward model.

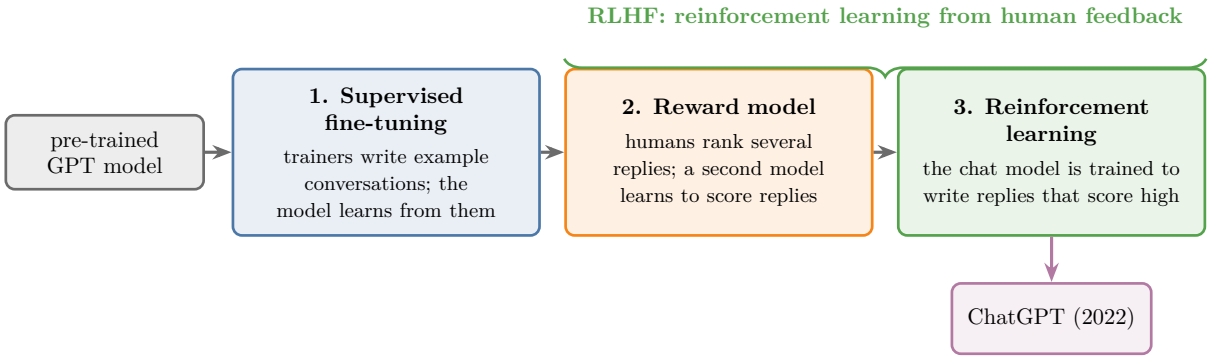


Figure 9: The three training steps of ChatGPT, after pre-training. Steps 2 and 3 form reinforcement learning from human feedback (RLHF)

Figure 9 shows the order: the human examples of step 1 come first, the human rankings of step 2 build the reward model, and step 3 uses that reward model.

Steps 2 and 3 together are **RLHF** (G-1695). Through the human rankings, the model learned to be more helpful and to refuse harmful requests, such as instructions for making weapons. Because ChatGPT is trained on dialogue, it can answer follow-up questions that refer to earlier parts of the conversation (OpenAI 2022). The thumbs-up and thumbs-down buttons in the app collect more feedback from users. In March 2023 OpenAI released GPT-4, a larger and more capable model (OpenAI 2023).

10. Summary

Stage	Year	Idea	Problem it left
1. Encoder–decoder	2014	LSTM encoder squeezes the input into a context vector; LSTM decoder writes the output	long sentences are forgotten
2. Attention	2014–15	a new context vector for every output word, mixing all encoder states	still one word at a time: slow
3. Transformer	2017	attention only, no RNN; all words in parallel	needs huge labelled data from scratch
4. Transfer learning	2018	pre-train a language model on unlabelled text, fine-tune on little data	still LSTM-based
5. LLMs	2018–	pre-trained transformer language models (GPT, BERT), scaled up into LLMs (GPT-2, GPT-3)	—
ChatGPT	2022	GPT fine-tuned on dialogue, then RLHF	—

- Sequence-to-sequence tasks map an input sequence to an output sequence of a different length, so the model cannot simply emit one output word per input word.
- Each stage of the history fixed the main problem of the stage before.
- **Language modelling** (G-1042) makes pre-training possible on any text, because the next word is its own label.
- "Large" means huge data, clusters of GPUs, weeks of training, high cost and high energy use, so only large companies, governments and large research institutes can train one.
- ChatGPT is a GPT model fine-tuned on dialogue and then with **RLHF** (G-1695), because a pre-trained language model only continues text and is not yet a helpful assistant.

11. Sources

Built from

- CampusX, "The Epic History of Large Language Models (LLMs) | From LSTMs to ChatGPT | CampusX", YouTube, https://www.youtube.com/watch?v=8fX3rOjTloc

- Sanderson, G. (3Blue1Brown), "Large Language Models explained briefly", 2024, 3blue1brown.com/lessons/mini-llm, https://www.youtube.com/watch?v=LPZh9BOjkQs. 0:01–0:33 (a chatbot as a script plus repeated next-word prediction); 3:13 (training compute as years of a computer doing a billion operations per second).
- Sanderson, G. (3Blue1Brown), "Transformers, the tech behind LLMs | Deep Learning Chapter 5", 2024, 3blue1brown.com/lessons/gpt, https://www.youtube.com/watch?v=wjZofJX0v4M. 5:42 (system prompt, user prompt, continuation).

Other references

- Sutskever, I., Vinyals, O. and Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *NeurIPS 2014*.
- OpenAI (2015). Introducing OpenAI. openai.com/blog, 11 December 2015.
- Bahdanau, D., Cho, K. and Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*. arXiv:1409.0473. Abstract; Figure 2 and section 5.2.1.
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. Section 1 and Table 2.
- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft, Chapter 7 "Transformers and Pretraining", §7.7 (self-supervised: the next word is its own label). https://web.stanford.edu/~jurafsky/slp3/7.pdf
- Howard, J. and Ruder, S. (2018). Universal Language Model Fine-tuning for Text Classification. *ACL 2018*. arXiv:1801.06146.
- Radford, A., Narasimhan, K., Salimans, T. and Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training. OpenAI. §4.1 (12 layers, 768-dimensional states, 12 heads; no parameter count).
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL 2019*. Section 3 (masked language model; BERT-Large has 340M parameters).
- Radford, A. et al. (2019). Language Models are Unsupervised Multitask Learners. OpenAI. §2.3 (the smallest model "is equivalent to the original GPT"); Table 2 (117M to 1,542M parameters).
- Brown, T. et al. (2020). Language Models are Few-Shot Learners. *NeurIPS 2020*. arXiv:2005.14165. Section 2 (175B parameters, 300 billion tokens, V100 cluster provided by Microsoft); section 2.2 (45 TB before and 570 GB after filtering); Appendix D, Table D.1 (GPT-3 175B: 3.14E+23 training FLOPs).
- OpenAI. GPT-2 small weights, `openai-community/gpt2` (model.safetensors and model card: "the smallest version of GPT-2, with 124M parameters"), huggingface.co.
- Patterson, D. et al. (2021). Carbon Emissions and Large Neural Network Training. arXiv:2104.10350. Table 4 (GPT-3: 10,000 V100 GPUs, 14.8 days, 1,287 MWh, 552 tCO_2e).
- U.S. Energy Information Administration (2023). How much electricity does an American home use? eia.gov/tools/faqs (10,791 kWh per residential customer in 2022).
- Ouyang, L. et al. (2022). Training language models to follow instructions with human feedback. *NeurIPS 2022*. arXiv:2203.02155. Figure 2 (the three steps).
- OpenAI (2022). Introducing ChatGPT. openai.com/blog, 30 November 2022.

- OpenAI (2023). GPT-4 Technical Report. arXiv:2303.08774.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Large language model (LLM) (G-1046)	A transformer language model with billions of parameters, trained on a vast amount of text.
Transformer (G-2007)	A neural network that turns one sequence into another (a seq2seq architecture) built from attention and dense layers, with no RNN, that processes all words in parallel.
Sequence-to-sequence (seq2seq) task (G-1772)	A task with a sequence as input and a sequence as output, possibly of different lengths, such as translation.
Context vector (G-461)	The summary of the input that the decoder works from; one fixed vector in the plain encoder–decoder, a new one per output word with attention.
Encoder (G-682)	The part of a seq2seq model that reads the input sequence and summarises it.
Decoder (G-564)	The part of a seq2seq model that writes the output sequence.
BLEU score (G-315)	A measure of translation quality: how many word sequences of a translation match a human reference.
Language modelling (G-1042)	Training a model to predict the next word of a text.
Self-supervised learning (G-2279)	Learning where the labels are made from the data itself, such as the next word of a text; it lets a model learn from any amount of unlabelled data, with no human labelling.
Masked language model (G-1170)	A pre-training task that hides some words and asks the model to predict them from both sides (BERT).
RLHF (G-1695)	Reinforcement learning from human feedback: improving a model with a reward model trained on human rankings of its outputs.

Term	Meaning
Pre-training (G-1546)	The first, general training of a model on a large dataset, such as ImageNet or a huge amount of unlabelled text, so that it learns general features; fine-tuning then adapts it to a task with little labelled data.
Attention (G-226)	A mechanism that lets each output step weigh all input positions and focus on the useful ones.
Transfer learning (G-2005)	Reusing a network trained by others on a big dataset for our own problem.
Machine translation (G-1141)	Translating a sentence from one language into another.
Sentiment analysis (G-1769)	Deciding whether a text expresses a positive or negative opinion.
Image captioning (G-918)	Producing a sentence that describes an image.
Long short-term memory (LSTM) (G-1123)	A recurrent network (RNN) that carries two memories from one time step to the next, a long-term one (the cell state) and a short-term one (the hidden state), with gates that control what each memory keeps and passes on.
Self-attention (intra-attention) (G-1763)	Attention in which the words of one sequence attend to each other (the queries, keys and values all come from that sequence); it gives each word a new vector that depends on the words around it.
Parameters (of a model) (G-1450)	The numbers inside a model that training learns from the data, such as a line's slope and intercept; once learned, they turn inputs into predictions.
Fine-tuning (G-779)	Training a pre-trained model further on a small dataset for a specific task.